Agentic Retrieval-Augmented Generation over Live Workspace Data: A Survey and the Cortex Assistant

Prof. Swapnali Gaikwad
Department of Computer Engineering
Dr. D. Y. Patil Institute of Technology
Pimpri, Pune, India
swapnali.gaikwad@dypvp.edu.in

Ajit Paraskar
Department of Computer Engineering
Dr. D. Y. Patil Institute of Technology
Pimpri, Pune, India
ajitparaskar87@gmail.com

Prathamesh Kumbhar
Department of Computer Engineering
Dr. D. Y. Patil Institute of Technology
Pimpri, Pune, India
prathameshkumbhar.dev@gmail.com

Harshvardhan Patil
Department of Computer Engineering
Dr. D. Y. Patil Institute of Technology
Pimpri, Pune, India
harsh.patil.tech@gmail.com

Bikram Ghosh
Department of Computer Engineering
Dr. D. Y. Patil Institute of Technology
Pimpri, Pune, India
ghoshbikram2725@gmail.com

***Abstract*—Retrieval-augmented generation (RAG) grounds a large language model (LLM) in retrieved text, but a single retrieve-then-read pass struggles with multi-hop questions, noisy passages and changing data. In agentic RAG the model controls retrieval itself. It can split a question, search more than once, pick a tool, judge its evidence and keep memory. We review this work around GRASP, which uses reinforcement learning to train a 3B-parameter policy that chooses, at each step, between semantic search, keyword search and reading a whole paragraph over a sentence-level index. We sort the literature into eight design dimensions and compare representative methods along them. As a case study we describe Cortex, an open-source assistant we built over Gmail, Google Calendar, Google Drive, Notion, Slack and GitHub. It plans sub-questions in one LLM call, searches in parallel, checks each cited claim against its evidence, keeps memories with provenance and expiry, and runs actions only after approval. On 12 questions over a synthetic workspace, the cited evidence held 92% of the expected key facts. Cortex declined to guess when no evidence existed, and no action ran without consent.**

***Index Terms*—Retrieval-augmented generation, agentic RAG, large language models, search policy, retrieval granularity, reinforcement learning, citation verification, agent memory, tool use, Model Context Protocol**

I. Introduction

An LLM's knowledge is frozen at training time and goes stale. RAG conditions the answer on passages fetched from an external corpus [1], usually in one pass, with a retriever ranking chunks and a generator reading the top few [2]. That works for one fact in a fixed collection. It fails when the second half of a question can only be searched after the first is answered [3], [4], when retrieved text is noisy enough to mislead [5], [6], and when the data keeps changing, as deadlines and issue states do [7].

In agentic RAG the model makes these calls itself. It decides whether to break the question up, when and how to search, whether it has enough evidence, and what is worth remembering [8]. Early methods relied on prompting [3], [9], [10]. Newer ones learn search behaviour with reinforcement learning (RL) [11], [12].

We build the review around GRASP (GRanularity-Aware Search Policy) [13]. Most systems fix the retrieval signal and the size of the text unit before any question arrives. GRASP learns both. Its searches return single sentences, and the agent reads the surrounding paragraph only when it asks to.

Our second subject is Cortex, an assistant we built over a small team's apps. It is not an implementation of GRASP. It plans once, searches in parallel and treats each source item as a fixed unit of evidence. Because we built it on free-tier models, rate limits shaped most of its design, and setting it beside GRASP helps show which research ideas survive in that kind of setting.

Contributions. Section III sets out an eight-dimension taxonomy and reads GRASP closely. Section IV compares representative methods, Section V gives the design of Cortex with early measurements, and Section VI lists open problems.

II. Background

A. Retrievers and Retrieval Units

BM25 ranks documents by term overlap and favours rare terms [14], so it handles exact names and identifiers well. Dense retrievers such as DPR embed questions and passages in one vector space [15] and can catch paraphrases. Static pipelines often run both and re-rank [2].

The indexed unit matters too. In Dense X, indexing short single-fact propositions instead of passages improved retrieval and QA at a fixed reader budget [16]. RAPTOR builds a tree of recursive cluster summaries for broad questions [17], and GraphRAG summarises communities in an entity graph [18]. In each case the unit is fixed at index time.

B. Limits of Single-Pass RAG

More context is not always better. Irrelevant sentences lower accuracy [5], and models underuse information placed in the

middle of long inputs [6]. Multi-hop benchmarks such as HotpotQA [19], 2WikiMultiHopQA [20] and MuSiQue [21] hide the second-hop entity until the first hop is solved, and Press *et al.* name the resulting failure the "compositionality gap" [4].

III. A Taxonomy of Agentic RAG

Fig. 1 groups the work into eight dimensions. A single system usually makes choices along several of them at once.

A. Query Planning and Decomposition

Self-Ask writes out follow-up questions and sends each to a search engine [4]. In Rewrite-Retrieve-Read a small rewriter is trained with RL, with the reader's output as feedback [22], while ReAct chooses every action only after seeing the previous observation [9]. A plan made up front is predictable and parallelises easily. A step-by-step plan adapts to results but costs extra calls.

B. Iterative and Adaptive Retrieval

IRCoT uses each chain-of-thought sentence as the next query [3]. FLARE retrieves when a drafted sentence contains low-confidence tokens [10], and Iter-RetGen queries with a whole draft answer [23]. Adaptive-RAG trains a classifier that picks no, single-step or multi-step retrieval by question complexity [24]. These methods vary when retrieval happens; the retriever and chunk size stay put.

C. Search Granularity and Search Policies

Search-R1 trains an LLM with RL to search in the middle of its reasoning, rewarding only the final answer [11], and Search-o1 adds search to large reasoning models [39]. MARAG-R1 trains an agent with semantic, keyword, filter and aggregate tools [12]. A-RAG exposes keyword search, semantic search and chunk reading as separate interfaces [25]. GRASP needs a closer look.

1) Problem: Gandhi *et al.* observe that RL-trained retrieval agents are normally bound to one retriever and to fixed chunks of fair size [13]. The kind and amount of evidence then cannot change between steps. GRASP folds three decisions into one learned policy: how to mix lexical with dense search, how much to read, and when to stop.

2) Formulation and actions: GRASP models the search loop as an MDP with a fixed turn limit. Its state holds the question, the history, the evidence and tools used so far, and the remaining turn budget. Each paragraph is split into sentences that point back to it. At each step the policy reasons and then takes one action: *semantic search* (Qwen3-0.6B embeddings), *keyword search* (BM25), *read paragraph* (the parent of a retrieved sentence) or *answer*. Searches return the top $K = 5$ sentences, so the agent spends context on a full paragraph only by choice.

TABLE I
Results reported by GRASP [13]: retrieval recall (R) and answer F1 on 500 validation questions per dataset. Compare the F1 columns: GRASP is best on all three, while IRCoT leads on HotpotQA recall. Numbers are copied from the GRASP paper, not re-run.

	HotpotQA		2Wiki		MuSiQue	
Method	R	F1	R	F1	R	F1
Single-step hybrid	0.86	0.48	0.60	0.32	0.59	0.20
IRCoT	**0.91**	0.36	0.83	0.40	0.62	0.15
Search-R1 (PPO)	0.76	0.47	0.74	0.48	0.55	0.26
Search-R1 (GRPO)	0.74	0.56	0.77	0.53	0.62	0.30
Base (no RL)	0.36	0.24	0.27	0.20	0.24	0.10
GRASP	0.90	**0.66**	**0.90**	**0.60**	**0.70**	**0.33**

3) Reward: The trajectory reward is

$$R = R_A + \alpha R_R + \beta R_S + \gamma R_E, \quad (\alpha, \beta, \gamma) = (0.7, 0.15, 0.15). \tag{1}$$

R_A is answer token F1. R_R is the F1 between paragraphs read and gold supporting paragraphs, which penalises reading distractors. R_S is 1 only if semantic and keyword search *each* find a gold paragraph. $R_E = (T - T_{\text{cur}})/T$, where T is the turn budget, rewards finishing early. It is paid only when $R_A > 0.5$, so a fast wrong answer earns nothing. The auxiliary weights sum to 1, which keeps answer F1 dominant and limits reward hacking.

4) Training and evaluation: Qwen2.5-3B-Instruct is fine-tuned with GRPO [40] on the HotpotQA distractor split [19] for 270 steps, only about 19% of an epoch. Episodes stop after 10 turns, and retrieved tokens are masked out of the loss. Testing uses 500 validation questions from each of HotpotQA, 2WikiMultiHopQA and MuSiQue [19]–[21]. The last two are never seen in training. Baselines are single-step lexical, semantic and hybrid retrieval, IRCoT driven by a proprietary model, the public Search-R1 PPO and GRPO checkpoints, and an untrained "Base" agent given GRASP's prompt and tools.

5) Findings: On all three datasets GRASP has the best F1, and it has the best recall on 2WikiMultiHopQA and MuSiQue (Table I). The authors put IRCoT's slightly higher HotpotQA recall down to the strong proprietary model writing its reasoning. Even single-step retrieval beats the untrained Base agent, so tools do little for a model that has not learned to use them. In a HotpotQA ablation, F1 falls only from 0.631 to 0.621 without keyword search and to 0.567 without semantic search, but removing the read action (searches then return whole paragraphs) brings it down to 0.484. The authors' reading is that a paragraph puts many entities in view at once, and the agent's next queries drift back toward the original question instead of following one clue.

6) Learned behaviour: A typical trajectory starts with semantic search. The agent reads one paragraph to pick up a bridge entity and runs a keyword search on it; if that fails, it widens to a broader semantic query. The authors liken this to skimming a page and then scanning it for a name.

7) Limitations stated by the authors: R_R and R_S need gold supporting facts, which few corpora have. The policy

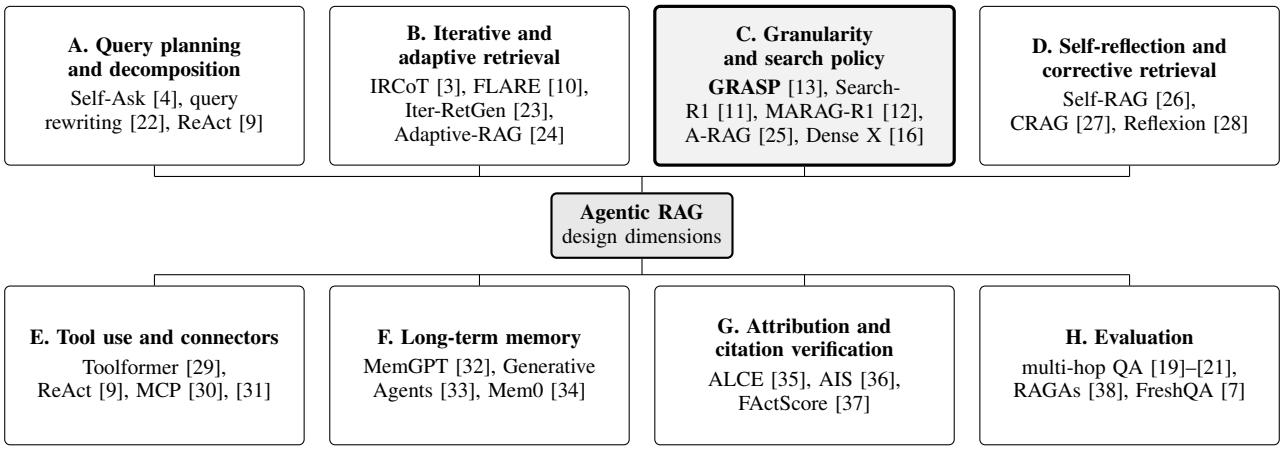


Fig. 1. The eight design dimensions used in this review, with representative (not exhaustive) methods for each. Branch C, shaded, holds the anchor work GRASP [13]. Some methods, such as ReAct, appear in two branches.

also sometimes answers too early, tool use plateaus near eight calls, and only one 3B backbone was tried. For future work they propose larger tool sets, rewards built from weaker or model-based grounding signals, and granularity that ranges from spans to whole document regions. Their ethics statement recommends provenance, citations, audit logs and human approval for consequential actions, all of which return in Section V.

D. Self-Reflection and Corrective Retrieval

Self-RAG learns reflection tokens that decide when to retrieve and grade relevance and support [26]. CRAG scores what was retrieved and switches to web search if the documents look poor [27], and Reflexion writes notes on past failures for the next attempt [28]. GRASP has no critic at inference time. The grounded-reading reward does that job, but only while the policy is trained.

E. Tool Use and Connector Protocols

Toolformer learns from a few examples where API calls help [29], and ReAct made interleaved reasoning and tool calls common [9]. Anthropic's Model Context Protocol (MCP), released in November 2024, standardises how clients discover and call external tools [30], but every server adds attack surface [31]. Indirect prompt injection becomes a concrete risk once retrieved text can trigger tool calls [41].

F. Long-Term Memory

MemGPT swaps data between the context window and external storage, much like OS paging [32]. Generative Agents score memories on recency, importance and relevance, then write reflections back [33]. Mem0 pulls facts out of conversations and consolidates them, optionally as a graph [34]. Once the data is personal, provenance, expiry and deletion cannot be afterthoughts.

G. Attribution and Citation Verification

ALCE tests whether cited passages support an LLM's statements and finds many that do not [35]. AIS defines attribution to identified sources for human raters [36], and FActScore scores atomic facts against a knowledge source [37]. Like Cortex, all three judge individual claims.

H. Evaluation

Most agentic RAG work, GRASP included, evaluates on Wikipedia multi-hop QA [19]–[21]. RAGAs scores faithfulness without reference answers [38], and FreshQA targets answers that change over time [7]. We found no benchmark for agents that work over private, changing, multi-source data and also take actions.

IV. Comparative Analysis

Table II compares representative methods. "Granularity" records whether the evidence unit is fixed or chosen by the agent, and "Live data" whether sources change at question time. A dash means the work does not address that dimension.

The learned search policies (Search-R1, MARAG-R1, GRASP) were all trained on static Wikipedia corpora with gold evidence, while the methods that query live sources use prompting. Only A-RAG and GRASP let the agent choose the evidence unit at query time, and GRASP's ablation is the most direct evidence we found that this choice changes accuracy. Memory, claim verification and actions appear in none of the learned-retrieval papers. They show up in deployed systems, where cost and side effects matter as much as benchmark scores.

V. Case Study: Cortex

Cortex answers questions such as "what changed on this project this week, and what is at risk?" over a user's connected apps. Every factual sentence cites numbered evidence, unsupported claims are struck through, conflicts between sources are listed, and requested actions wait for approval. We describe only what is implemented.

TABLE II

SURVEYED METHODS AND CORTEX ALONG THE TAXONOMY DIMENSIONS, AS DESCRIBED IN EACH PUBLICATION. READ DOWN A COLUMN TO SEE WHICH DIMENSIONS ARE RARELY COMBINED: NO METHOD WITH A LEARNED POLICY ALSO HAS MEMORY, LIVE DATA OR ACTIONS.

Method	Retrieval strategy	Granularity	Policy	Memory	Verification	Live data	Actions
RAG [1]	Single-shot dense retrieval	Fixed passage	Fine-tuned retriever and generator	–	–	–	–
Self-Ask [4]	Explicit follow-up sub-questions	Search-engine answers	Prompting	–	–	Web search	–
IRCoT [3]	Retrieve at each reasoning step	Fixed paragraph	Prompting	–	–	–	–
FLARE [10]	Retrieve when draft has low confidence	Fixed passage	Prompting	–	Regenerate uncertain sentence	–	–
Adaptive-RAG [24]	Route: none, single or multi-step	Fixed	Trained classifier	–	–	–	–
Self-RAG [26]	On demand via reflection tokens	Fixed passage	Fine-tuned	–	Relevance and support tokens	–	–
CRAG [27]	Evaluator-triggered correction	Strip filtering	Trained evaluator	–	Retrieval evaluator	Web fallback	–
ReAct [9]	Interleaved thought and action	API page or lookup	Prompting	–	–	Wikipedia API	Environment actions
Search-R1 [11]	Multi-turn search in reasoning	Fixed passage	RL (PPO, GRPO)	–	–	–	–
MARAG-R1 [12]	Semantic, keyword, filter, aggregate tools	Fixed chunk	SFT, then RL	–	–	–	–
A-RAG [25]	Hierarchical retrieval interfaces	Keyword to chunk	Model-driven tool use	–	–	–	–
GRASP [13]	Coordinated semantic and keyword search	Sentence, paragraph on demand	RL (GRPO), 4-part reward	–	Grounded-reading reward (training only)	–	–
MemGPT [32]	Paging between context and storage	–	Function calls	Tiered virtual context	–	–	–
Mem0 [34]	Memory extraction and retrieval	–	–	Consolidated, graph variant	–	–	–
Cortex (ours)	LLM plan of 2–5 sub-questions, parallel per-source search, one bounded retry	Fixed per source item	Rules, prompted router and planner, hard budgets	Salience, TTL, decay, pin, edit, forget	Claim-level lexical check, LLM re-check, conflicts	Six apps, web, MCP servers	Proposed; run only after approval

A. Architecture

Cortex has two modes. Deep mode plans sub-questions and asks an LLM to re-check doubtful claims. Fast mode skips both steps and saves those calls. Fig. 2 shows one Deep-mode run. The orchestrator is a hand-written loop rather than a graph framework, so every branch is visible in the code and every run has a fixed ceiling: at most 5 sub-questions, 18 tool calls, 16 evidence items (8 in Fast mode), 90 s, and 5 LLM calls in Deep mode or 3 in Fast mode.

Check and recall. An unavailable source is reported with the reason, and the run continues without it. The memory service returns up to five memories. Pinned ones are always included, and the rest are ranked by relevance times $0.5+0.5\times$ salience.

Route. Regular expressions handle clear cases at no model cost: greetings, "remember that…", imperatives such as "email…", and app names that narrow the search. Other questions cost one small-model call that returns a route (general, personal, web or action), the sources and the model tier. This follows Adaptive-RAG [24], with rules and prompting in place of a trained classifier. It routed all 20 of our test questions correctly.

Plan. In Deep mode one fast-tier LLM call returns JSON with a rewritten question, an optional time window and 2–5 sub-questions, each naming 1–3 sources and keywords. Invalid JSON, Fast mode and the action route fall back to one sub-question over all in-scope sources.

Search and rank. All (source, query) pairs run in parallel with a 12 s timeout, one retry and a 90 s cache. A sub-question that finds nothing gets one broader retry at no LLM cost. Evidence is deduplicated and ranked by keyword overlap plus a recency bonus. If nothing is found, Cortex says so and stops without calling a model.

Synthesize. The model may use only the numbered evidence and must cite every factual sentence. It then lists conflicts, gaps, proposed actions, memory candidates and follow-up questions.

Verify. A cited sentence is *supported* if at least 40% of its content words, and every number and date, appear in the evidence it cites. Between 20% and 40% it is *partial*, and below 20% it is *unsupported*. Hedges and advice are skipped. In Deep mode one LLM call re-checks up to eight doubtful claims. Groundedness is the mean claim score, with partial support counted as half.

Actions. Proposed actions are validated against the connector's schema, stored with an exact preview and shown as a confirmation card. Only the approval endpoint executes them. Each approval is single-use, bound to one user and expires after 30 minutes, so model text never reaches a write API directly.

Memory. A rules gate discards questions, small talk and anything that looks like a secret. Otherwise one fast-model call proposes at most two memories, each with a kind (priority, preference, fact, person or decision), a salience and a reason shown to the user. Explicit requests are stored at salience 0.95, other candidates only at 0.5 or above. A contradiction replaces an unpinned memory, and a near-duplicate reinforces it. In a

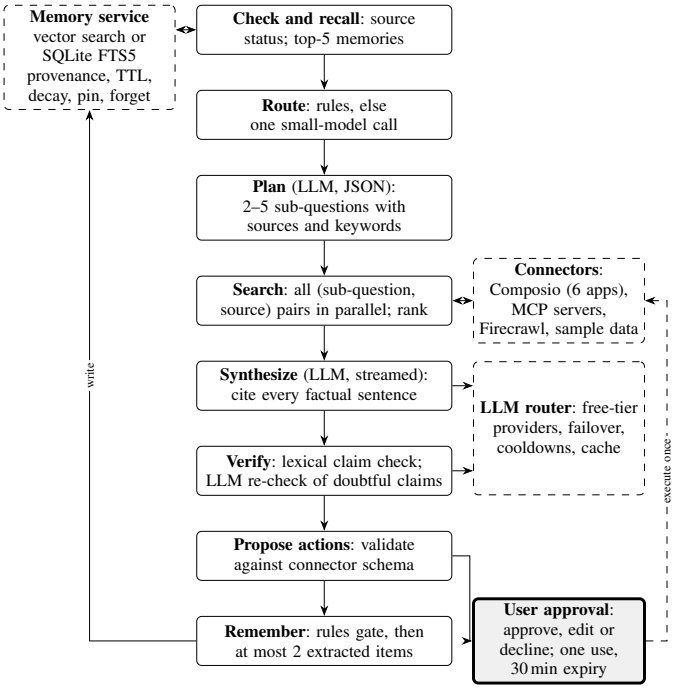


Fig. 2. One Cortex run in Deep mode, top to bottom. Solid boxes are pipeline steps; dashed boxes are services they call. The only path to a connector's write action runs through the shaded approval box, which needs a separate request from the user. If search finds no evidence, the run stops after *Search* without an LLM call.

TABLE III
CORTEX DESIGN CHOICES, THE WORK EACH DRAWS ON, AND HOW GRASP RELATES. THE RIGHT COLUMN SHOWS WHERE A GRASP IDEA COULD REPLACE A FIXED RULE IN CORTEX.

Cortex choice	Related work	Relation to GRASP
Up-front plan of sub-questions	Self-Ask [4], rewriting [22]	GRASP decides step by step; Cortex plans once for predictable cost.
Parallel search, one retry	IRCoT [3], Adaptive-RAG [24]	Multi-turn search could replace the fixed retry when a hop depends on an earlier result.
Item-level evidence	Dense X [16], A-RAG [25]	Fixed unit; GRASP's ablation argues for sentence hits plus an explicit "read" action.
Keyword queries to each source's own search	BM25 [14], DPR [15]	One signal per source; GRASP's complementary-search idea applies.
Hard budgets	FLARE [10], Adaptive-RAG [24]	Play the role of GRASP's turn-efficiency term, but fixed rather than learned.
Claim verification, conflicts	ALCE [35], AIS [36], FActScore [37], CRAG [27]	Could supply the weak grounding signal GRASP lists as future work.
"Not found" answer, list of gaps	Self-RAG [26]	Addresses GRASP's stated failure mode of answering too early.
Governed memory	MemGPT [32], Gen. Agents [33], Mem0 [34]	Outside GRASP's scope.
MCP connectors, approval gate	ReAct [9], MCP [30], [31], injection [41]	Follows GRASP's ethics advice on human approval for consequential actions.

scripted 30-message test this stored all 12 expected memories and no secrets. Priorities expire after 45 days, facts after 90 and decisions after 120, while preferences and people do not expire. Salience halves every 30 days unless pinned, and users can edit or delete any memory.

Oracle Agent Memory stores the memory text and serves vector search with a 0.35 distance cut-off. A SQLite ledger holds provenance, pins, expiry and decay, caps unpinned memories at 200 per user, and provides FTS5 keyword search when Oracle is not configured.

Connectors and models. Gmail, Google Calendar, Google Drive, Notion, Slack and GitHub connect through Composio. MCP servers and Firecrawl web search are configurable, and a fictional sample workspace with seeded conflicts is used for demos and evaluation. The LLM router tries free-tier providers in a fixed order (Groq, Gemini, Cerebras, OpenRouter, then Puter). A provider that hits a rate limit is put on cooldown, responses are cached, and a streamed answer switches provider only if no token has arrived yet.

Implementation. A TypeScript monorepo shares one pipeline across the Next.js app, the tests and the evaluation scripts. The memory service runs on FastAPI.

B. Mapping Cortex to the Literature

Table III links each design choice to prior work and to GRASP.

Planning and retrieval. One up-front plan, close to Self-Ask with rewriting [4], [22], fits free-tier limits and keeps latency low. The price is that no sub-question can use another's result, which is the bridge-entity step IRCoT and GRASP rely on [3], [13].

Where GRASP applies. Four GRASP ideas carry over. *Read on demand.* Cortex passes each source item to the synthesizer whole or truncated, so a long email thread costs a lot of context. The read-action ablation argues for returning the best-matching sentences with a pointer to the parent item and fetching the full thread only when a snippet looks relevant but incomplete. *Complementary signals.* Connectors mostly rely on each app's keyword search. An embedding index over fetched items would add a semantic signal, although workspace questions full of issue numbers and names may favour keywords more than Wikipedia does. *Learned budgets.* Cortex's hard limits play the role of GRASP's turn-efficiency term but cannot adapt to question difficulty. *Weak rewards.* Cortex has no gold evidence, but its verifier labels each claim against specific evidence, which is one candidate for the model-based grounding signal GRASP proposes. A policy trained on those labels could learn to please a lexical checker without becoming more correct, so human spot checks would be needed.

Beyond GRASP's setting. GRASP works on a static Wikipedia corpus. Cortex's sources fail independently, their facts go stale, they contradict each other, and the agent can change them. So Cortex reports each outage, filters by time window, lists conflicts instead of silently picking a side, and gates execution on user approval, which also limits injected instructions [31], [41].

TABLE IV
PRELIMINARY CORTEX MEASUREMENTS (12 QUESTIONS, FICTIONAL SAMPLE WORKSPACE, FREE-TIER MODELS, ONE RUN PER VERSION). LOOK AT THE ACTION AND LATENCY ROWS FOR THE EFFECT OF THE ROUTER; THE GROUNDEDNESS VALUES USE DIFFERENT CHECKERS AND SHOULD NOT BE COMPARED. THIS IS NOT A BENCHMARK COMPARISON.

Metric	First	+ Router
Mean groundedness (lexical + LLM re-check)	0.88	0.75
Mean citation recall	0.92	0.92
Honest "not found" answers	1/1	1/1
Cross-source conflicts detected	1/2	1/2
Expected actions proposed	1/2	2/2
Actions executed without consent	0	0
Median latency	3.7 s	4.6 s
90th-percentile latency	7.1 s	6.3 s

C. Preliminary Measurements

Table IV reports two Deep-mode runs of Cortex's 12-question evaluation over its sample workspace: the first pipeline, and a later version with the router and a stricter checker. Both used free-tier models (Groq `openai/gpt-oss-120b`, with Gemini 3.5 Flash-Lite as fallback). Citation recall is the share of expected key facts present in the cited evidence.

These numbers are limited. We wrote the questions, the workspace is synthetic, and each version ran once. Groundedness comes from Cortex's own, partly lexical verifier, and the second run used a stricter checker, so the drop from 0.88 to 0.75 does not by itself mean answers got worse. In the first run rate limits sent 8 of 12 questions to Gemini, so model choice is confounded. No other system ran on the same data. Still, citation and verification worked end to end, Cortex said "not found" instead of guessing, no action ran without approval, and a separate action route raised correct proposals from 1/2 to 2/2. Conflict detection found only one of the two seeded conflicts, because it depends on the synthesizer noticing the disagreement. We have not built a dedicated conflict component yet.

VI. OPEN CHALLENGES AND FUTURE DIRECTIONS

Supervision without gold evidence. RL search policies need gold supporting facts [12], [13]. Verifier labels, agreement between trajectories, or AIS- and FActScore-style scores [36], [37] could substitute, with human checks against gaming.

Granularity over heterogeneous data. What the "sentence" and its "parent" should be in an email thread, a pull request or a calendar event is still open. Hierarchical units [17], [25] and propositions [16] give two places to start.

Freshness and conflict. Time-sensitive questions remain hard [7], and few methods ever show a conflict to the user. Most treat it as noise to filter out.

Verification quality. Claim checkers should be calibrated against human attribution judgements [35], [36] and should report uncertainty.

Safety and memory. Approval gates reduce injection risk without removing it [31], [41], and long-term memory [32]–[34] raises questions of consent and deletion that no benchmark measures.

Cost-aware evaluation. GRASP settles near eight tool calls [13], but a deployed system must also budget LLM calls, much as Adaptive-RAG budgets retrieval depth [24]. Benchmarks over private, changing data should score citation support, abstention, conflicts and action safety, not only accuracy.

VII. CONCLUSION

GRASP shows that how much an agent reads, and which retrieval signal it uses, can be learned. Its 3B model beat prompted and RL-trained baselines on answer F1. Cortex instead runs over live, heterogeneous sources under tight budgets, with a fixed retrieval plan and its effort spent on verification, conflict reporting, governed memory and approval-gated actions. Combining the two needs rewards without gold evidence, evidence units for mail, chat and code, and evaluations of trust and safety.

REFERENCES

[1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. tau Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474.

[2] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, Q. Guo, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," arXiv preprint arXiv:2312.10997, 2023.

[3] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions," in *Proc. 61st Annu. Meeting of the Association for Computational Linguistics (ACL)*, 2023, pp. 10 014–10 037.

[4] O. Press, M. Zhang, S. Min, L. Schmidt, N. A. Smith, and M. Lewis, "Measuring and narrowing the compositionality gap in language models," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023, pp. 5687–5711.

[5] F. Shi, X. Chen, K. Misra, N. Scales, D. Dohan, E. H. Chi, N. Schärli, and D. Zhou, "Large language models can be easily distracted by irrelevant context," in *Proc. 40th Int. Conf. Machine Learning (ICML)*, 2023.

[6] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024.

[7] T. Vu, M. Iyyer, X. Wang, N. Constant, J. Wei, J. Wei, C. Tar, Y.-H. Sung, D. Zhou, Q. Le, and T. Luong, "FreshLLMs: Refreshing large language models with search engine augmentation," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 13 697–13 720.

[8] A. Singh, A. Ehtesham, S. Kumar, and T. T. Khoei, "Agentic retrieval-augmented generation: A survey on agentic RAG," arXiv preprint arXiv:2501.09136, 2025.

[9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023.

[10] Z. Jiang, F. F. Xu, L. Gao, Z. Sun, Q. Liu, J. Dwivedi-Yu, Y. Yang, J. Callan, and G. Neubig, "Active retrieval augmented generation," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 7969–7992.

[11] B. Jin, H. Zeng, Z. Yue, J. Yoon, S. Arik, D. Wang, H. Zamani, and J. Han, "Search-R1: Training LLMs to reason and leverage search engines with reinforcement learning," in *Proc. Conf. Language Modeling (COLM)*, 2025, arXiv:2503.09516.

[12] Q. Luo, X. Li, Y. Wang, T. Fan, Y. Li, X. Chen, and X. Qiu, "MARAG-R1: Beyond single retriever via reinforcement-learned multi-tool agentic retrieval," arXiv preprint arXiv:2510.27569, 2025.

[13] V. Gandhi, J. Lee, S. Todmal, F. Dernoncourt, R. Rossi, Z. Wang, and A. Lan, "GRASP: GRanularity-Aware Search Policy for agentic RAG," arXiv preprint arXiv:2607.10463, 2026, listed by Adobe Research as appearing in Findings of EMNLP 2026.

[14] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[15] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. tau Yih, "Dense passage retrieval for open-domain question answering," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781.

[16] T. Chen, H. Wang, S. Chen, W. Yu, K. Ma, X. Zhao, H. Zhang, and D. Yu, "Dense X retrieval: What retrieval granularity should we use?" in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2024, pp. 15 159–15 177.

[17] P. Sarthi, S. Abdullah, A. Tuli, S. Khanna, A. Goldie, and C. D. Manning, "RAPTOR: Recursive abstractive processing for tree-organized retrieval," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.

[18] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, and J. Larson, "From local to global: A graph RAG approach to query-focused summarization," arXiv preprint arXiv:2404.16130, 2024.

[19] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. W. Cohen, R. Salakhutdinov, and C. D. Manning, "HotpotQA: A dataset for diverse, explainable multi-hop question answering," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2018, pp. 2369–2380.

[20] X. Ho, A.-K. D. Nguyen, S. Sugawara, and A. Aizawa, "Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps," in *Proc. 28th Int. Conf. Computational Linguistics (COLING)*, 2020, pp. 6609–6625.

[21] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "MuSiQue: Multihop questions via single-hop question composition," *Transactions of the Association for Computational Linguistics*, vol. 10, pp. 539–554, 2022.

[22] X. Ma, Y. Gong, P. He, H. Zhao, and N. Duan, "Query rewriting in retrieval-augmented large language models," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 5303–5315.

[23] Z. Shao, Y. Gong, Y. Shen, M. Huang, N. Duan, and W. Chen, "Enhancing retrieval-augmented large language models with iterative retrieval-generation synergy," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023, pp. 9248–9274.

[24] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. C. Park, "Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity," in *Proc. Conf. North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024, pp. 7036–7050.

[25] M. Du, B. Xu, C. Zhu, S. Wang, P. Wang, X. Wang, and Z. Mao, "A-RAG: Scaling agentic retrieval-augmented generation via hierarchical retrieval interfaces," arXiv preprint arXiv:2602.03442, 2026.

[26] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024.

[27] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, "Corrective retrieval augmented generation," arXiv preprint arXiv:2401.15884, 2024.

[28] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[29] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023, pp. 68 539–68 551.

[30] Anthropic, "Introducing the Model Context Protocol," https://www.anthropic.com/news/model-context-protocol, Nov. 2024.

[31] X. Hou, Y. Zhao, S. Wang, and H. Wang, "Model context protocol (MCP): Landscape, security threats, and future research directions," arXiv preprint arXiv:2503.23278, 2025.

[32] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," arXiv preprint arXiv:2310.08560, 2023.

[33] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proc. 36th Annu. ACM Symp. User Interface Software and Technology (UIST)*, 2023.

[34] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav, "Mem0: Building production-ready AI agents with scalable long-term memory," arXiv preprint arXiv:2504.19413, 2025.

[35] T. Gao, H. Yen, J. Yu, and D. Chen, "Enabling large language models to generate text with citations," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 6465–6488.

[36] H. Rashkin, V. Nikolaev, M. Lamm, L. Aroyo, M. Collins, D. Das, S. Petrov, G. S. Tomar, I. Turc, and D. Reitter, "Measuring attribution in natural language generation models," *Computational Linguistics*, vol. 49, no. 4, pp. 777–840, 2023.

[37] S. Min, K. Krishna, X. Lyu, M. Lewis, W. tau Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, and H. Hajishirzi, "FActScore: Fine-grained atomic evaluation of factual precision in long form text generation," in *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, 2023, pp. 12 076–12 100.

[38] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAs: Automated evaluation of retrieval augmented generation," in *Proc. 18th Conf. European Chapter of the Association for Computational Linguistics: System Demonstrations (EACL)*, 2024, pp. 150–158.

[39] X. Li, G. Dong, J. Jin, Y. Zhang, Y. Zhou, Y. Zhu, P. Zhang, and Z. Dou, "Search-o1: Agentic search-enhanced large reasoning models," arXiv preprint arXiv:2501.05366, 2025.

[40] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo, "DeepSeekMath: Pushing the limits of mathematical reasoning in open language models," arXiv preprint arXiv:2402.03300, 2024.

[41] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 79–90.